

5.4 Control de formularis

L'entrada de dades a través de formularis per part dels usuaris pot provocar problemes a l'aplicació web si no es filtren prèviament aquestes dades.

Per aconseguir-ho, haurem d'aplicar alguna de les següents polítiques:

1. Data validation (Validació de dades)

- Es quan ens hem d'assegurar que les dades que agafem a través del formulari siguin del tipus que esperem o tinguin el format correcte.
- Per exemple:
 - Si demanem un número enter a l'usuari, assegurar-nos que és enter
 - Si demanem un telèfon, no pot tenir lletres.
 - Si és una matrícula de cotxe ha de ser de 4 números i 3 lletres
 - Si és un NIF ha de respectar el format...
- Per assegurar-ho, tenim diverses funcions:
 - `is_integer(...)` → per saber si és un número enter (i totes les variants amb els altres tipus)
 - `strlen(...)` → per controlar la mida
 - `preg_match('expressió_regular', $variable)` → per controlar el format d'una variable a través d'una expressió regular (regex)
 - `filter_var(...)` → per a comprovar si una variable compleix amb algun format. Hi ha molts tipus de filtres:
`FILTER_VALIDATE_EMAIL, FILTER_VALIDATE_IP, FILTER_VALIDATE_URL`

Exemple: filtrar matrícula de cotxe amb `preg_match`

```
if (preg_match('/^\d{4}[A-Za-z]{3}$/', $matricula)) {
    echo "La matrícula $matricula es válida";
} else {
    echo "La matrícula $matricula NO es válida";
}
```

- `/^` → inici del string
- `\d{4}` → 4 digits (equivalent a `[0-9]{4}`)
- `[A-Za-z]{3}` → 3 lletres majúscules o minúscules
- `$/` → final de l'string

Exemple: filtrar correu electrònic amb `filter_var`

```
$email = "ejemplo@dominio.com";
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "CORREU OK";
}else{
    echo "CORREU INCORRECTE";
}
```

2. Data sanitization

- És quan hem d'editar i manipular les dades que ens passa l'usuari pel formulari, per assegurar-nos que són segures, fins i tot és possible que haguem d'esborrar certes parts.
- La funció més habitual és —> `strip_tags(...)` que permet eliminar les etiquetes HTML que conté un string

Exemple 1:

```
<?php
    if(isset($_POST["boto"])){
        $dadaFiltrada=strip_tags($_POST["dada"]);
        echo "DADA FILTRADA: ".$dadaFiltrada;
    }
?>
<form method="POST">
    <input type="text" name="dada">
    <input type="submit" name="boto">
</form>
```

—> Si ara l'usuari posa etiquetes HTML en la caixa de text, aquestes queden eliminades.

Exemple 2: Què passa si injectem codi javascript en la caixa anterior?

Si en una caixa de text l'usuari tecleja `<script>alert("hacked");</script>` amb `strip_tags(...)` aconseguim que s'elimini l'etiqueta HTML → `<script> </script>`
D'aquesta manera no s'executarà l'script que l'usuari ha intentat injectar, i simplement el contingut de l'script es mostrarà com a text

Exemple: strip_tags(HTML) + preg_match(MATRICULA)

```
<?php
    if(isset($_POST["boto1"])){
        $dadaFiltrada=strip_tags($_POST["dada"]);
        echo "DADA FILTRADA amb strip_tags: ".$dadaFiltrada;
        echo "<br>";

        $matricula=$_POST["matricula"];
        if (preg_match('/^\d{4}[A-Za-z]{3}$/', $matricula)) {
            echo "Matricula OK: $matricula";
        }else{
            echo "Matricula INCORRECTA: $matricula";
        }
    }
?>
<form method="POST">
    <input type="text" name="dada" placeholder="HTML">
    <input type="text" name="matricula" placeholder="1234ABC">
    <input type="submit" name="boto1">
</form>
```

- També es poden sanititzar els formularis amb la funció `htmlspecialchars(...)` per a prevenir HTML i script injection. Amb aquesta funció s'aconseguirà no executar el possible codi que hagi teclejat l'usuari en la caixa de text, sinó que s'interpretarà literalment com a text i no com a codi executable.

Exemple 1 – ús de la funció `htmlspecialchars(...)`

```
$input = "<b>Hello</b> <script>alert('hack');</script>";
echo htmlspecialchars($input);
```

Per pantalla mostrarà:

```
&lt;b&gt;Hello&lt;/b&gt; &lt;script&gt;alert('hack');&lt;/script&gt;
```

Què mostrarà el següent codi?

```
<?php
    if(isset($_POST["boto2"])){
        //echo $_POST["dada"];
        $dada_filtrada = htmlspecialchars($_POST["dada"]);
        echo $dada_filtrada;
    }
?>

<form method="POST">
    <input type="text" name="dada" placeholder="<script>alert('hacked');</script>"
    <input type="submit" name="boto2">
</form>
```

→ Si l'usuari tecleja el que hi ha en el placeholder, no s'executarà cap script ja que `htmlspecialchars` ha escapat el codi HTML de tal manera que es mostrarà literalment el què ha escrit l'usuari, però no s'executarà.

→ Hi ha variants de `htmlspecialchars` amb més paràmetres per a ser encara més rigor:

```
$nom = htmlspecialchars(strip_tags($_POST["nom"]), ENT_QUOTES, 'UTF-8');
```

- A l'hora de fer consultes a una base de dades, podem utilitzar la funció `mysqli_real_escape_string(..)`
Aquesta funció protegeix els caràcters especials com per exemple cometes simples i cometes dobles, de tal manera que ajudarà a evitar atacs d'injecció SQL.

Incís:**Enviar formulari sense submit:**

A qualsevol element de formulari se li pot afegir l'atribut `onchange` i assignar-li l'acció d'apretar el submit.

`<input type="radio" name="borrar" onchange="this.form.submit()" value=..... >`

Exemple

```
<form action="" method="post">
  <input type="radio" name="borrar" value="<?php echo $value;|?>" onchange="this.form.submit()">
</form>
<br>
```